Discharge Parametric Studies

Release 1.0

André Kapelrud, Robert Marskar

Mar 03, 2026

CONTENTS

1 Introduction **3**
1.1 Basic Usage 3
1.2 Overall Design 4
1.3 Run Definition 4

2 Writing jobscripts **15**
2.1 Writing Jobscripts 15

3 Prerequisites **25**
3.1 Required Software 25
3.2 Local Slurm Installation (Debian/Ubuntu) 26

This page contains documentation for a collection of python and bash/slurm scripts for running multilevel chombo-discharge studies over wide parameter spaces.

CHAPTER

ONE

INTRODUCTION

1.1 Basic Usage

The `configurator.py` script can be used to set up directory structures and submit slurm jobs for wide parametric sweeps over chombo-discharge based studies.

Listing 1: Invocation options $ python configurator.py –help

```
usage: configurator.py [-h] [--verbose] [--logfile LOGFILE] [--output-dir OUTPUT_DIR]
                       [--dim DIM]
                       run_definition

Batch script for running user-defined, parametrised chombo-discharge studies.

positional arguments:
  run_definition        parameter space input file. Json read directly, or if .py file␣
↪look
                        for 'top_object' dictionary

options:
  -h, --help            show this help message and exit
  --verbose             increase verbosity
  --logfile LOGFILE     log file. (Postfix) Rotated automatically each invocation.
  --output-dir OUTPUT_DIR
                        output directory for study result files
  --dim DIM             Dimensionality of simulations. Must match chombo-discharge
                        compilation.
```

The most important part is the `run_definition`, which is either structured as a json file or python file. If it is a python file it is dynamically imported and the object `top_object` (a dictionary) is loaded. A python dictionary containing basic types (sub-dictionaries, lists, strings, numbers, booleans) almost has the same syntax structured JSON data, and it is easy to use the json module to dump such a dictionary to a json file if needed. There are benefits for keeping your *run_definition* as a .py file though, e.g. the possibility to use variables when setting up the project structure or specifying numerical parameter ranges.

1.2 Overall Design

This piece of software parses a run definition dictionary/JSON structure that is comprised of *databases* and *studies*.

Listing 2: config_concept.py

```
db_study = {
    ...
}

main_study = {
    ...
}

top_object = dict(
        databases=[db_study],
        studies=[main_study]
        )
```

The difference between a *database* and a *study* is mainly a semantic one; a study can depend on database, and the configurator will create and submit a slurm job hierarchy as well as create an output file hierarchy that reflects this.

A database is meant to be used as a first simulation step running specific (perhaps light-weight) jobs (chombo-discharge simulations, or other software) to generate intermediate data that the main studies depend on.

Each database or study step relies on running a (chombo-discharge) executable repeatedly, and often in parallel, over a defined parameter set. Ex.: if a parameter *pressure* should be varied over 5 different values, and another parameter *radius* should be varied over 3 different values, the parameter space would require 15 distinct runs. Now, the preliminary database study might only depend on the *pressure*, while the whole main study depends on both parameters. This would result in 5 jobs (submitted as parallel array job to slurm) for the database study, **followed** by 15 jobs (parallel, and depending on the success of the first database's array job) for the main study.

1.3 Run Definition

A database/study will have these configurable fields:

- `identifier`, unique
- `output_directory`, relative to the cmdline output directory, i.e. `--output-dir`
- `output_dir_prefix` (default: `"run_"`)
- `program`, executable to run. Will be copied to output directory hierarchy.
- `job-script`, job script to do the heavy lifting (call mpirun / start simulation code)
- `job_script_dependencies`, (python) scripts and files that the job-script needs
- `required_files`, typical input files for the simulation, post-analysis scripts, etc.
- `parameter_space`, a dictionary of parameters (c.f. *Defining Parameter Spaces*)

The difference between `job_script_dependencies` and `required_files` is that the `required_files` will be copied into the bottom-level directory where the program is run from, i.e. into every specific *run* directory for every invocation over the parameter space.

- `required_files` is typically used for `*.inputs` chombo-discharge files, physical/chemical input files like `*.json` files, or other plain data files. Sometimes extra python modules or bash-scripts might be needed at the run-level, so use this field to copy those dependencies in.
- `job_script_dependencies`, as the name implies, should point to whatever code is needed to configure and submit the actual slurm jobs.

The `configurator.py` script will set up directory structures and copy files into place, then launch slurm array jobs over all the configured parameter space referenced in each database. Then the same is repeated for the second-level studies. These second-level slurm array jobs are made dependent on the database jobs, essentially securing that they are run in sequence.

Important: It is up to the *job_script* to submit the actual slurm jobs. Facilities are in place to translate a given slurm array job id/index into the correct parameter values.

Listing 3: directory structure example

```
import numpy as np

inception_stepper = {
    'identifier': 'inception_stepper',
    'output_directory': 'is_db',
    'program': 'program{DIMENSIONALITY}d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex',
    'job_script': 'discharge_inception_jobscript.py',
    'job_script_dependencies': [
        'generic_array_job.sh',
        'parse_report.py',
        ...
        ],
    'required_files': [
        'master.inputs',
        'transport_data.txt'
        ],
    'parameter_space': {
        ...
        }
    }

plasma_study = {
    'identifier': 'photoion',
    'output_directory': 'study0',
    'program': 'program{DIMENSIONALITY}d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex',
    'job_script': 'plasma_jobscript.py',
    'job_script_dependencies': [
        'generic_array_job.sh',
        ...
        ],
    'required_files': [
        'master.inputs',
        'chemistry.json',
        'detachment_rate.dat',
        'electron_transport_data.dat'
        'Analyze.py',
```

(continues on next page)

(continued from previous page)

```
        ],
    'parameter_space': {
        ...
        }
    }

top_object = dict(
        databases=[inception_stepper],
        studies=[plasma_study]
        )
```

The example above has a more realistic structure. How the parameter spaces are defined can be found in a later section.

Note the use of a templated filename for the `program` field, where the part `"{DIMENSIONALITY}"` is substituted with the dimension specified on the command line using the `--dim` flag.

Just after issuing this command, when the first slurm job for the database named *'inception_stepper'* has just started in the subdirectory `run_0`, the resulting file hierarchy from this could look like:

```
$ ls -R --file-type output-dir
.:
is_db/  study0/

./is_db:
array_job_id                      jobscript_symlink@                                run_
↪0/
discharge_inception_jobscript.py  master.inputs                                      ␣
↪structure.json
generic_array_job.sh              parse_report.py                                    ␣
↪transport_data.txt
index.json                        program3d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex

./is_db/run_0:
chk/    geo/           mpi/             plt/    pout.1  pout.3  program@  restart/
crash/  master.inputs  parameters.json  pout.0  pout.2  pout.4  regrid/   transport_data.
↪txt

./study0:
Analyze.py                   generic_array_job.sh  parse_report.py
array_job_id                 inception_stepper@    plasma_jobscript.py
chemistry.json               index.json            program3d.Linux.64.mpic++.gfortran.
↪OPTHIGH.MPI.ex
config_util.py               jobscript_symlink@    run_0/
detachment_rate.dat          json_requirement.py   structure.json
electron_transport_data.dat  master.inputs

./study0/run_0:
Analyze.py      detachment_rate.dat          generic_array_job.sh  parameters.json
chemistry.json  electron_transport_data.dat  master.inputs         program@
```

Do notice:

- The rather self-explainatory named `jobscript_symlink` symlink pointing to the jobscripts:

```
output-dir/is_db$ readlink jobscript_symlink
discharge_inception_jobscript.py

output-dir/study0$ readlink jobscript_symlink
plasma_jobscript.py
```

- The `study0/inception_stepper` symlink pointing across the file hierarchy:

```
output-dir/study0$ readlink inception_stepper
../is_db
```

- The `program` symlinks in the *"run_"** sub-directories. These point to the actual executable in their respective parent directories.

```
output-dir/is_db/run_0$ readlink program
../program3d.Linux.64.mpic++.gfortran.OPTHIGH.MPI.ex
```

- A job-script typically receives an array job index from slurm (through the environment variable `$SLURM_ARRAY_TASK_ID`), and must use this to find the relevant parameters, dependent databases, get structural metadata and enter its own run-subdirectory and execute code there.
- For each database/study there are certain metadata files that are generated to make it possible to programatically traverse the created file-hierarchy from within the jobscripts or from within post-simulation analysis scripts. These files becomes especially imortant when the second-level studies have to traverse the databases' result hierarchies to retrieve and parse database results before launching their own slurm jobs.

  Note: Sometimes, one need to manipulate simulation input files directly from the job-scripts, e.g. to change some parameter depending on a *database* result. Python utility functions are provided to manipulate configuration files on-the-fly in this intermediate step. C.f. examples in the *Database-dependent jobscript examples* for usage.

  Generated files:

  - `array_job_id` containing a single integer; the slurm array job id for the this database/study
  - `index.json`, containing a mapping between specific array job indices and all parameter sets for this database/study
  - `structure.json` a parsed dump of the overall structure of the database/study. This matches a parsed export of the corresponding section in the original *run_definition*. Included both for data consistency and for usage by the jobscripts to get extra metadata for setting up batch jobs.
  - `run_*/parameters.json` containing the actual parameter space point for that run.

The *configurator.py* script contains helper code to in-place manipulate both *.*inputs* files (normally used to specify chombo-discharge parameters), as well as generic structured *.json* files (e.g. used by chombo-discharge or physical/chemical data input.

1.3.1 Defining Parameter Spaces

A parameter space is a dictionary where the uniquely named keys (also dictionaries) are the parameters.

```
"parameter_space" = {
         "parameter_name_0":{
             ...
         },
         "another_descriptive_parameter_name":{
             ...
         },
         "some_other_parameter_name":{
             ...
         }
     }
```

Each parameter can contain several fields:

- `database`: unique identifier referencing a database study. This specifies that this parameter is used in a database, and hence that this study depends on the referenced database
- `target`: the file target (either `*.json` or `*.inputs` files))
- `uri`: (abbr. of *Uniform Resource Identifier*); the address to the resource you want to change within the target file.
- `values`: `list()/[]` of values. Scalars should be written as `[scalar-value]`. Depending on the uri, this can be a 2nd order list (list of lists) if the uri references several uri endpoints. In that case the parameter will change two or more uri targets at the same time.

Continuing the example from the previous section:

Listing 4: directory structure example cont…

```
import numpy as np

inception_stepper = {
    'identifier': 'inception_stepper',
    'output_directory': 'is_db',
    ..
    'parameter_space': {
        "pressure": {
            "target": "master.inputs",
            "uri": "DischargeInception.pressure"
            },
        "geometry_radius": {
            "target": "master.inputs",
            "uri": "Rod.radius",
            },
        }
    }

plasma_study = {
    'identifier': 'photoion',
    'output_directory': 'study0',
    ...
    'parameter_space': {
```

(continues on next page)

(continued from previous page)

```
        "geometry_radius": {
            "database": "inception_stepper",  # database dependency
            "target": "master.inputs",
            "uri": "Rod.radius",
            "values": [1e-3, 2e-3, 3e-3]
            },
        "pressure": {
            "database": "inception_stepper",  # database dependency
            "target": "chemistry.json",
            "uri": ["gas", "law", "ideal_gas", "pressure"],
            "values": np.arange(1e5, 11e5, 1e5).tolist()
            },
        "photoionization": {
            "target": "chemistry.json",
            "uri": [
                "photoionization",
                [
                    '+["reaction"=<chem_react>"Y + (O2) -> e + O2+"]',  # non-optional␣
↪match
                    '*["reaction"=<chem_react>"Y + (O2) -> (null)"]'  # optional match␣
↪(create-if-not-exists)
                    ],
                "efficiency"
                ],
            "values": [[1.0, 0.0]]  #[[float(v), float(1.0-v)] for v in np.arange(0.0, 1.
↪0, 1.0)]
            },
        }
    }

top_object = dict(
        databases=[inception_stepper],
        studies=[plasma_study]
        )
```

The actual parameter values are typically specified for the top study definition, not the databases. The database only specifies where changes are to be made for a given value.

In this example there are several distinctly named parameters changing different aspects of the simulations:

- `geometry_radius`:

  Marked as dependent on the database; meaning that this study will run after the database study.

  Important: Both the database and the study has the same `target` file name and `uri` field, namely the `master.inputs` chombo-discharge input file and its `Rod.radius` field. Note that these are now different files residing within the output directory file hierarchy.

  This parameter contributes a factor 3 to the overall parameter space size.

- `pressure`:

  List of values (Evaluates to `[100000.0, 200000.0, ..., 900000.0, 1000000.0]`).

  The database will write this parameter to the `master.inputs` file by changing the

DischargeInception.pressure (uri-field) input parameter, while the study has a different target, utilizing the json-writing capabilities to change the pressure field in the json hierarchy according to the list in the uri-field.

This parameter contributes a factor 10 to the overall parameter space size.

- photoionization:

  This is the most complex parameter. It only affects the chemistry.json target in the study.

  As can be seen in the uri specification the second level is a list of length 2. This means that this parameter changes two fields in chemistry.json at the same time. Similarly, the values field is a double list, where the first element of each contained list will be written to the uri of the first target field and the second element of each contained list will be written to the uri of the second target field.

  This, the value [[1.0, 0.0]] is to be regarded as a scalar quantity in the parameter space, and this parameter contributes a factor 1 to the overall parameter space size (not really affecting it).

  The resulting change for the runs in *study0/run_*/chemistry.json* will be:

Listing 5: resulting chemistry.json

```
{
    ...
    "photoionization": [
        {
            "reaction": "Y + (O2) -> e + O2+",
            "efficiency": 1.0
        },
        {
            "reaction": "Y + (O2) -> (null)",
            "efficiency": 0.0
        }
    ],
    ...
}
```

For the special syntax encountered see *Navigating JSON object hierarchies*.

Navigating JSON object hierarchies

The special syntax +["field-name"="search-value"] and *["field-name"="search-value"] is used to search a json list for an child object {...} containing a specific member "field-name" with a specific value "search-value".

- +[] requires the object with the member field-name to exist.
- *[] will create the object with the member field-name if it doesn't exist.

Note: If searching for an object in a list where the search key itself is an JSON object, search-value can be omitted: +["fields-name"]. Remember to repeat the fields-name in your uri as the next list element to select that child object if needed.

```
{
    "parent":{
```

(continues on next page)

(continued from previous page)

```
        "list-1":[
            {
                "field-name-0":"value_0"
            },
            {
                "field-name-1":{
                    "target-field":"change-me!"
                }
            },
            {
                "field-name-2":"value_2"
            },
        ]
    }
}
```

which can be found with

```
"uri" = [
    "parent",
    "list-1",
    '+["field-name-1"]',  # this finds the "unnamed" container object
    "field-name-1", # selects the object
    "target-field"  # this is the actual target within the above object
    ]
```

Note: The special notation `<chem_react>` is a hint to the parser that the value searched for in this specific example should be a valid chombo-discharge chemical reaction, c.f. "Specifying reactions" in the Plasma Model. The comparison of the chemical reactions between `search-value` and json file is thus a parsed/semantic comparison.

Navigating a json object hierarchy can sometimes involve having to search through several lists down the tree:

```
{
    "parent":{
        "list-level-1":[
            {
                "field-name-0":"value_0"
            },
            {
                "field-name-1":"value_1_0",
                "target-field":"dont-you-change-me!"
            },
            {
                "field-name-1":"value_1_1",
                "target-field":"change-me!"
            },
            {
                "field-name-2":"value_2"
            },
        ]
    }
```

(continues on next page)

(continued from previous page)

```
}
```

In the above contrived example, we want to change the third contained object in the list; i.e. the object that has `"field-name-1":"value_1_1"`. The required parameter space uri would be:

```
"uri" = [
    "parent",
    "list-level-1",
    '+["field-name-1"="value_1_1"]',  # this finds the object
    "target-field"  # this is the actual target within the above object
    ]
```

A deeper hierarchy with two list levels to traverse:

```
{
    "parent":{
        "list-level-1":[
            {
                "field-name-0":"value_0"
            },
            {
                "field-name-1":"value_1_0",
                "target-field":"dont-you-change-me!"
            },
            {
                "field-name-1":"value_1_1",
                "target-field":[
                    {
                        "search-field"="some-value",
                        "target2-field":"don-try-to-change-me!"
                    },
                    {
                        "search-field"="search-value",
                        "target2-field":"change-me!"
                    }
                ]
            },
            {
                "field-name-2":"value_2"
            },
        ]
    }
}
```

```
"uri" = [
    "parent",
    "list-level-1",
    '+["field-name-1"="value_1_1"]',  # find the right object
    "target-field",  # alter this one
    '+["search-field"="search-value"]',  # find the right object
    "target2-field"  # target aquired!
    ]
```

The corresponding value specification for this parameter in the run_definition should be a single list: `"values"=[new-value-0, new-value-1, ..., new-value-N]` contributing a factor N to the parameter space size.

Dummy parameters

It is possible to pass `dummy` parameters as a mechanism to set options for the jobscripts. A dummy parameter doesn't have to specify a target file, only a name and `values`-field, and optionally the `database` field. If the values is a single element the parameter won't grow the parameter space size, and thus not contributing to an increase in the number of slurm jobs. A dummy parameter will end up in the generated `index.json`, `structure.json` and `parameters.json` files.

Say, if study's jobscript needs a configurable parameter we can use a dummy parameter to pass it:

```
db_study = {
    ...
    "parameter_space": {
        ...  # no "K_min" parameter here
    }
}

main_study = {
    ...
    "parameter_space": {
        "K_min": {
            "values": [6.0]
            },
    }
}

top_object = dict(
        databases=[db_study],
        studies=[main_study]
        )
```

CHAPTER

TWO

WRITING JOBSCRIPTS

2.1 Writing Jobscripts

configurator.py schedules slurm jobs through *sbatch* expecting the bash script `generic_array_job.sh`. This should edited to suit the system configuration, and added as a `job_script_dependency` to the studies:

Listing 1: generic_array_job.sh

```
#!/bin/bash
# Author André Kapelrud
# Copyright © 2025 SINTEF Energi AS

#SBATCH --account=<cluster account>
#SBATCH --output=R-%x.%A-%a.out
#SBATCH --error=R-%x.%A-%a.err

# typical options needed for running on cluster
# remove leading '#' to use
##SBATCH --nodes=4 --ntasks-per-node=128
##SBATCH --partition=normal
##SBATCH --time=0-00:10:00

# Local slurm testing,
# add extra '#' to comment out
#SBATCH --ntasks=5 --cpus-per-task=1
#SBATCH --time=0-00:25:00

set -o errexit
set -o nounset

# example sigma2, module loading code
if command -v module > /dev/null 2>&1
then
    module restore system
    module load foss/2023a
    module load HDF5/1.14.0-gompi-2023a  # needed by chombo-discharge
    module load Python/3.11.3-GCCcore-12.3.0  # needed by jobscripts
fi

python ./jobscript_symlink  # run python through job-script
exit $?
```

The script configures the resource requirements, sets error conditions and loads sigma2 system modules (c.f. Lmod).

Note: It is possible to submit python scripts to sbatch directly if the python script has the correct shebang (`#! /usr/bin/env python`), the `#SBATCH`-specific comment directives also works from a python script. Routing through an intermediate bash-script made it somewhat easier to configure module-loading on sigma2.

A simple way of having two different versions of this bash script is to just make `generic_array_job.sh` a symlink to a correct system-tailored version, or even copy it in similar to the way chombo-discharge deals with the library makefiles.

2.1.1 Template jobscripts

At this stage the work is not done, because alot of of the heavy lifting has to be done by your jobscripts. Regard the *configurator.py* script as setting up the infrastructure. It is now up to you to start meaningful simulations. This section gives some examples on how to accomplish this.

Generic python jobscript example

A vanilla, quite simple python-based jobscript might look like this:

Listing 2: generic_array_job_jobscript.py

```python
#!/usr/bin/env python
"""
Author André Kapelrud
Copyright © 2025 SINTEF Energi AS
"""

import os
import sys
import json
import re
import logging
import subprocess

# local imports
sys.path.append(os.getcwd())  # needed for local imports from slurm scripts
from config_util import get_slurm_array_task_id

if __name__ == '__main__':

    log = logging.getLogger(sys.argv[0])
    formatter = logging.Formatter(
            '%(asctime)s | %(levelname)s :: %(message)s')
    sh = logging.StreamHandler(sys.stdout)
    sh.setFormatter(formatter)
    log.addHandler(sh)
    log.setLevel(logging.INFO)

    task_id = get_slurm_array_task_id()
    log.info(f'found task id: {task_id}')
```

(continues on next page)

(continued from previous page)

```

    with open('index.json') as index_file:
        index_dict = json.load(index_file)
    job_prefix = index_dict['prefix']

    dpattern = f'^({job_prefix}[0]*{task_id:d})$'  # account for possible leading zeros
    dname = [f for f in os.listdir() if (os.path.isdir(f) and re.match(dpattern, f))][0]
    log.info(f'chdir: {dname}')
    os.chdir(dname)

    input_file = None
    for f in os.listdir():
        if os.path.isfile(f) and f.endswith('.inputs'):
            input_file = f
            break

    if not input_file:
        raise ValueError('missing *.inputs file in run directory')

    cmd = f"mpirun program {input_file} Random.seed={task_id:d}"
    log.info(f"cmdstr: '{cmd}'")
    p = subprocess.Popen(cmd, shell=True, executable="/bin/bash")

    while True:
        res = p.poll()
        if res is not None:
            break
```

Database-dependent jobscript examples

This section contains two example jobscripts.

- One for a database where the simulation (chombo discharge) is rerun under some condition.
- One for a study that needs to extract some dataset from a database and set up sub directories per parameter space run.

The jobscripts depend on two python scripts: `parse_report.py` and `config_util.py` that needs to be included as `job_script_dependencies` in the *run_definition*.

> **Warning:** These are not *ready-to-run*, but illustrates a concept. For specific examples see the actual source code listings of this project.

Listing 3: Example database (python) jobscript

```
#!/usr/bin/env python
"""
Author André Kapelrud
Copyright © 2025 SINTEF Energi AS
"""

```

(continues on next page)

(continued from previous page)

```python
import os
import sys
import json
import re
import logging
import subprocess
import time
import math
import shutil

# local imports
sys.path.append(os.getcwd())  # needed for local imports from slurm scripts
from parse_report import parse_report_file  # noqa: E402
from config_util import (  # noqa: E402
                         get_slurm_array_task_id,
                         handle_combination,
                         read_input_float_field
                         )

if __name__ == '__main__':

    log = logging.getLogger(sys.argv[0])
    formatter = logging.Formatter(
            '%(asctime)s | %(levelname)s :: %(message)s')
    sh = logging.StreamHandler(sys.stdout)
    sh.setFormatter(formatter)
    log.addHandler(sh)
    log.setLevel(logging.INFO)

    task_id = get_slurm_array_task_id()
    log.info(f'found task id: {task_id}')

    with open('index.json') as index_file:
        index_dict = json.load(index_file)

    # extract the directory prefix of run directories (default is 'run_',
    # but make no assumptions.
    job_prefix = index_dict['prefix']

    # find the directory corresponding to this array task id
    dpattern = f'^({job_prefix}[0]*{task_id:d})$'  # account for possible leading zeros
    dname = [f for f in os.listdir() if (os.path.isdir(f) and re.match(dpattern, f))][0]

    # step into directory
    log.info(f'chdir: {dname}')
    os.chdir(dname)

    # find chombo-discharge *.inputs file
    input_file = None
    for f in os.listdir():
        if os.path.isfile(f) and f.endswith('.inputs'):
            input_file = f
```

(continues on next page)

(continued from previous page)

```
            break

    if not input_file:
        raise ValueError('missing *.inputs file in run directory')

    # We are now ready to run mpi on our chombo-discharge executable
    # through the program symlink If there are any quirks specific to this
    # invocation that is not taken care of in your *.inputs file, you can add
    # them here:

    cmd = f"mpirun program {input_file} Random.seed={task_id:d} SomeNamespace.variable=QuirkSolution"
    log.info(f"cmdstr: '{cmd}'")
    p = subprocess.Popen(cmd, shell=True, executable="/bin/bash")
    while p.poll() is None:
        time.sleep(0.5)
    # propagate nonzero exit code to calling jobscript
    if p.returncode != 0:
        sys.exit(p.returncode)

    # First simulation step done.
    # We are free to do whatever is necessary here. One likely scenario is
    # to parse some results, alter some parameters and rerun the invocation
    # above.

    result_fn = "result-file-name"

    def parse_some_result_file(result_filename):
        """ meaningless stub """
        return None

    data = parse_some_result_file(result_fn)
    log.info("Some description of this step...")

    def calculate_interresting_value(data):
        """ meaningless stub """
        return None  # stub
    iv = calculate_interresting_value(data)

    # If we need to read something from a *.inputs file we can of course do that:
    orig_iv = read_input_float_field(input_file, 'SomeNamespace.interrestingvariable')
    if orig_iv None:
        raise RuntimeError(f"'{input_file}' does not contain 'SomeNamespace.interrestingvariable' field")

    # some decision point
    if orig_iv != iv:
        log.info(f'renaming: {result_fn} -> {result_fn}.0')
        shutil.move(result_fn, f'{result_fn}.0')

        # You might want to back up other results here.

```

(continues on next page)

(continued from previous page)

```
        # update input file, this mirrors the run_definition file syntax
        handle_combination({
            "interresting_parameter": {  # parameter name
                "target": input_file,  # file target
                "uri": "SomeNamespace.interrestingvariable"
                }
            }, dict(interresting_parameter=iv))

        # rerun the simulation!
        log.info('Rerunning calculations')
        p = subprocess.Popen(cmd, shell=True, executable="/bin/bash")
        while p.poll() is None:
            time.sleep(0.5)
        sys.exit(p.returncode)
```

Study (database-dependent) jobscript example

This is a rather long example where we traverse the database directories to find relevant data and then set up detailed simulations to use that data. This can be a single simulation, or a complex-subhierarchy of simulations. The last part is only pseudo-code, so the reader is advised to check out some of the checked in example studies in the main repository.

```
#!/usr/bin/env python
"""
Author André Kapelrud
Copyright © 2025 SINTEF Energi AS
"""

import os
import sys
import json
import re
import logging
import itertools
import shutil

from subprocess import Popen, PIPE

from pathlib import Path

# local imports
sys.path.append(os.getcwd())  # needed for local imports from slurm scripts
from parse_report import parse_report_file  # noqa: E402
from config_util import (  # noqa: E402
                         copy_files, backup_file,
                         get_slurm_array_task_id,
                         handle_combination,
                         DEFAULT_OUTPUT_DIR_PREFIX
                         )


if __name__ == '__main__':
```

(continues on next page)

(continued from previous page)

```

    log = logging.getLogger(sys.argv[0])
    formatter = logging.Formatter(
            '%(asctime)s | %(levelname)s :: %(message)s')
    sh = logging.StreamHandler(sys.stdout)
    sh.setFormatter(formatter)
    log.addHandler(sh)
    log.setLevel(logging.INFO)

    task_id = get_slurm_array_task_id()
    log.info(f'found task id: {task_id}')

    with open('structure.json') as structure_file:
        structure = json.load(structure_file)

    # extract the directory prefix of run directories (default is 'run_',
    # but make no assumptions.
    job_prefix = 'run_'
    if 'output_dir_prefix' in structure:
        job_prefix = structure['output_dir_prefix']

    dpattern = f'^({job_prefix}[0]*{task_id:d})$'  # account for possible leading zeros
    dname = [f for f in os.listdir() if (os.path.isdir(f) and re.match(dpattern, f))][0]
    log.info(f'chdir: {dname}')
    os.chdir(dname)

    # locate .inputs file (should be in the required_files list, and copied to the
    # current directory):
    input_file = None
    for f in os.listdir():
        if os.path.isfile(f) and f.endswith('.inputs'):
            input_file = f
            break
    if not input_file:
        raise ValueError('missing *.inputs file in run directory')
    log.info(f"input file: {input_file}")

    # get access to structure of dependent database through symlink
    with open('../inception_stepper/structure.json') as db_structure_file:
        db_structure = json.load(db_structure_file)

    # determine order of parameters in database (might differ from the order in this study)
    if 'space_order' not in db_structure:
        raise ValueError("missing field 'space_order' in database 'inception_stepper'")
    db_param_order = db_structure['space_order']

    # load this run's parameters (radius, pressure, etc.)
    with open('parameters.json') as param_file:
        parameters = json.load(param_file)

    # we can add run-time checks:
```

(continues on next page)

(continued from previous page)

```
    if 'geometry_radius' not in parameters:
        raise RuntimeError("'geometry_radius' is missing from 'parameters.json'")

    # put the parameters in the same order as the database index needs them
    db_search_index = []
    for db_param in db_param_order:
        db_search_index.append(parameters[db_param])

    # Now, we need to locate the corresponding data in the 'database':
    with open('../inception_stepper/index.json') as db_index_file:
        db_index = json.load(db_index_file)

    # linear search through index, which is a dictionary.
    index = -1
    for db_i, params in db_index['index'].items():
        if params == db_search_index:
            index = int(db_i)
            break
    if index < 0:
        raise RuntimeError(f'Unable to find db parameter_set: {db_param_order} = ' +
                           f'{db_search_index}')
    log.info(f"Found database parameters {db_param_order} = {db_search_index} "
             f"at index: {index}")

    # we have the index, now locate the correct subdirectory:
    db_run_path = Path('../inception_stepper')
    if 'prefix' in db_index:
        db_run_path /= db_index['prefix'] + str(index)
    else:
        db_run_path /= DEFAULT_OUTPUT_DIR_PREFIX + str(index)


    def parse_and_get_interresting_data(filename):
        """ stub """
        return None

    data = parse_and_get_interresting_data(db_run_path / '<some-database-result-file>')

    # Maybe we need to do some selective picking of data based on a dummy-parameter?
    # here we can for easily check a 'dummy' parameter
    if 'dummy-parameter' in parameters:
        data = some_filter_action(parameters['dummy-parameter']  #do something meaningful

    #-----------------------------------------------------------------------------
    # At this point we can do whatever we like with the data.

    # Maybe the database study gave an estimate of a parameter and we just
    # want to write that parameter to an *.inputs file or *.json file and run
    # a detailed simulation.
    # If so, use the handle_combination() function to write the data and
    # launch the job using mpirun. See database example.

```

(continues on next page)

(continued from previous page)

```
    # If on the other hand, the database produces e.g. a voltage list and some
    # other associated input data, then we need to create a sub-file
    # hierarchy in the run-directory and submit those simulation jobs to slurm.

    # We utilize helper functions from the configurator to alleviate the burden.

    # We will use the generic_array_job_jobscript.py script at the leaf directory level.
    #-----------------------------------------------------------------------------

    # let us enumerate the interresting data, assuming some known structure:
    #   data[i] corresponds to the (new) parameters ["voltage", "some-other-parameter"]
    enum_table = list(enumerate(data))

    output_prefix = "voltage_"

    # first we have to create an index for the sub-directories:

    # guard for reposting of the job
    MAX_BACKUPS = 10
    index_path = Path('index.json')
    backup_file(index_path, max_backups=MAX_BACKUPS)

    # write voltage index
    with open(index_path, 'w') as voltage_index_file:
        json.dump(dict(
            key=["voltage", "some-other-parameter"],
            prefix=output_prefix,
            index={i: item for i, item in enum_table}
            ),
                  voltage_index_file, indent=4)

    # recreate the generic job-script symlink, so that the actual .sh jobscript work:
    if not os.path.islink('jobscript_symlink'):
        os.symlink('generic_array_job_jobscript.py', 'jobscript_symlink')

    # create run directories, copy files, set voltage and parameters, etc.
    for i, row in enum_table:
        voltage_dir = Path(f'{output_prefix}{i:d}')
        # don't delete old invocations
        backup_dir(voltage_dir, max_backups=MAX_BACKUPS)
        os.makedirs(voltage_dir, exist_ok=False)

        # further symlink program executable to this directory's program-symlink
        link_path = voltage_dir / 'program'
        if not link_path.is_symlink():
            os.symlink(Path('../program'), link_path)

        # grab original file names from structure
        required_files = [Path(f).name for f in structure['required_files']]
        copy_files(log, required_files, voltage_dir)

        # reuse the combination writing code from the configurator / config_util, by
```

(continues on next page)

(continued from previous page)

```
        # building a fake combination and parameter space:
        # populate values
        comb_dict = dict(
                voltage=row[0],
                some_other_parameter=row[1]
                )
        pspace = {
                "voltage": {
                    "target": voltage_dir/input_file,
                    "uri": "SomeNamespace.potential",
                    },
                "some_other_parameter": {
                    "target": voltage_dir/'chemistry.json',
                    'uri': [ ... ]  # some very complex JSON traversing uri
                    },
                }
        handle_combination(pspace, comb_dict)

    # all voltage_* directories are now ready, and we can post a (new!) slurm array job:
    cmdstr = f'sbatch --array=0-{len(enum_table)-1} ' + \
            f'--job-name="{structure["identifier"]}_voltage" ' + \
            'generic_array_job.sh'
    log.debug(f'cmd string: \'{cmdstr}\'')
    p = Popen(cmdstr, shell=True, stdout=PIPE, encoding='utf-8')

    job_id = -1
    while True:  # wait until sbatch is complete
        # try to capture the job id
        line = p.stdout.readline()
        if line:
            m = re.match('^Submitted batch job (?P<job_id>[0-9]+)', line)
            if m:
                job_id = m.groupdict()['job_id']

                array_job_id_path = Path('array_job_id')
                # backups for previously posted runs:
                backup_file(array_job_id_path, max_backups=MAX_BACKUPS)

                # write array index file
                with open(array_job_id_path, 'w') as job_id_file:
                    job_id_file.write(job_id)
                log.info(f"Submitted array job (for '{structure['identifier']}" +
                         f"_voltage' combination set). [slurm job id = {job_id}]")

        if p.poll() is not None:
            break
```

CHAPTER

THREE

PREREQUISITES

3.1 Required Software

3.1.1 Slurm

Slurm (Simple Linux Utility for Resource Management) is an open-source workload manager and job scheduler for Linux clusters. It handles queuing, resource allocation, and execution of batch jobs across one or more compute nodes.

This project uses Slurm to:

- Submit array jobs (`sbatch --array`) — each array index maps to one point in the parameter space
- Enforce job dependencies — database jobs must complete successfully before dependent study jobs are allowed to start
- Supply `$SLURM_ARRAY_TASK_ID` to jobscripts so they can locate their own parameter set and run directory

Slurm has three main components:

- `slurmctld` — the central controller daemon; manages the job queue and scheduling decisions
- `slurmd` — the compute node daemon; executes jobs as directed by the controller
- Client tools (`sbatch`, `squeue`, `sinfo`, `scancel`) — used to submit and monitor jobs

For local testing, both `slurmctld` and `slurmd` can run on the same machine — a workstation or WSL instance is sufficient for small test runs. See *Local Slurm Installation (Debian/Ubuntu)* for a step-by-step setup guide.

On a cluster, Slurm is typically pre-installed and managed by system administrators. You need only the client tools and access to an account and partition.

3.1.2 Python

configurator.py and its helper modules require Python >= 3.13.0. Only standard library modules are used by the configurator itself.

Certain example analysis scripts in the repository additionally require:

- `numpy`

3.2 Local Slurm Installation (Debian/Ubuntu)

This guide covers installing Slurm on a single Debian or Ubuntu machine for local testing, including WSL. Both the controller (`slurmctld`) and compute node (`slurmd`) will run on the same host.

Important: If running in WSL, ensure the `memory` setting in your wsl-config (either `/etc/wsl.conf` per-distribution or `%homepath%\\.wslconfig` globally) is set high enough to match the `RealMemory` you will configure in `slurm.conf`, plus a margin for the OS itself.

3.2.1 Step 1 — Install packages

```
sudo apt-get update
sudo apt-get install -y munge libmunge-dev slurmctld slurmd slurm-client
```

Munge provides credential authentication between the controller and compute nodes. A munge key is generated automatically during installation.

3.2.2 Step 2 — Start Munge

Munge must be running before the Slurm daemons start:

```
sudo systemctl start munge
sudo systemctl status munge
```

3.2.3 Step 3 — Configure slurm.conf

Slurm is configured through `/etc/slurm/slurm.conf`. Two values must match your machine: the hostname and the hardware spec of the node.

Get your hostname:

```
hostname -s
```

Get the hardware spec that Slurm should use for the node (run as root so it can read all hardware info):

```
sudo slurmd -C
```

This prints a `NodeName=...` line with the correct `CPUs`, `RealMemory`, etc. for your machine. Use this line verbatim in the config below.

A minimal `/etc/slurm/slurm.conf` for a single-node setup:

Listing 1: /etc/slurm/slurm.conf

```
ClusterName=localcluster
SlurmctldHost=my-hostname          # replace with output of: hostname -s

MpiDefault=none
ProctrackType=proctrack/linuxproc
TaskPlugin=task/none
```

(continues on next page)

(continued from previous page)

```
SchedulerType=sched/backfill
SelectType=select/cons_tres
SelectTypeParameters=CR_Core

SlurmctldLogFile=/var/log/slurm/slurmctld.log
SlurmdLogFile=/var/log/slurm/slurmd.log
StateSaveLocation=/var/spool/slurmctld
SlurmdSpoolDir=/var/spool/slurmd

# Replace with the NodeName line from: sudo slurmd -C
NodeName=my-hostname CPUs=4 RealMemory=7800 State=UNKNOWN

PartitionName=debug Nodes=ALL Default=YES MaxTime=INFINITE State=UP
```

Note: `SlurmctldHost` and the `NodeName` hostname must both match the output of `hostname -s` exactly, or the daemons will fail to register with each other.

3.2.4 Step 4 — Create required directories

```
sudo mkdir -p /var/log/slurm /var/spool/slurmctld /var/spool/slurmd
sudo chown -R slurm:slurm /var/log/slurm /var/spool/slurmctld /var/spool/slurmd
sudo touch /var/log/slurm/slurmctld.log /var/log/slurm/slurmd.log
sudo chown slurm:slurm /var/log/slurm/slurmctld.log /var/log/slurm/slurmd.log
```

3.2.5 Step 5 — Start Slurm daemons

```
sudo systemctl start slurmctld
sudo systemctl start slurmd
```

Check that both are running:

```
sudo systemctl status slurmctld slurmd
```

3.2.6 Step 6 — Verify

```
$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*       up   infinite      1   idle my-hostname
```

The node should show as `idle`. If it shows `down` or `drain`, check `/var/log/slurm/slurmd.log` for errors — the most common cause is a hostname mismatch between `SlurmctldHost`/`NodeName` and the actual machine hostname.

You can now submit test jobs with `sbatch`.

3.2.7 Step 7 — Enable at boot (optional)

```
sudo systemctl enable munge slurmctld slurmd
```